

元学习（一）：Learning to Learn

把机器学习的三步骤提升到“学习算法本身”的层次



基于李宏毅老师《机器学习 2025》课程内容整理

2026 年 6 月 9 日

视频信息

频道：李宏毅深度学习课堂（Bilibili）

时长：约 46 分 20 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=55>

目录

1 为什么需要元学习	3
1.1 本章小结	4
2 从普通机器学习三步骤到元学习三步骤	4
2.1 本章小结	5
3 Step 1: 学习算法里什么可以被学	6
3.1 本章小结	7
4 Step 2: 如何评价一个学习算法	7
4.1 本章小结	8
5 Step 3: 优化学习算法本身	9
5.1 本章小结	10
6 Few-shot 框架中的任务、资料与循环	10
6.1 Support set 与 query set	10
6.2 Within-task 与 across-task	11
6.3 Loss 层级: 样本求和与任务求和	12
6.4 Episode 与 inner/outer loop	13
6.5 本章小结	14
7 总结与延伸	14
7.1 本章小结	16

1 为什么需要元学习

Meta Learning 常被翻译为“元学习”，也常被概括成 learning to learn。课程一开始先解释这个主题为什么自然：很多机器学习实践并不只是“把数据喂给模型然后结束”，而是充满大量人工选择，例如网络架构、学习率、优化器、正则化强度、数据增强方式、训练轮数等。普通深度学习系统看起来很自动，实际上常常需要人反复试超参数。

Source of image: <https://medium.com/intuitionmachine/the-brute-force-method-of-deep-learning-innovation-58b497323ae5> (Denny Britz's graphic)



感謝 沈昇勳 同學提供圖檔

图 1: 课程用“调参作业”引入元学习：深度学习系统的表现高度依赖网络结构、学习率等 hyperparameters；如果这些选择长期依赖人工经验，就说明“学习如何学习”本身也可以被系统化。¹

传统做法中，研究者或工程师会根据经验决定一个学习算法的大量细节，再让这个算法从训练数据中学出模型。元学习反过来问：能否让机器也参与决定“如何学”？也就是说，不只让模型学参数，还让系统学习产生模型的程序、策略或初始化。

“Meta” 在这里不是玄学

这里的 meta 不是指模型比普通模型更神秘，而是指研究对象提高了一个层级。普通机器学习找的是某个任务上的函数 f ，元学习找的是一个能够产生函数的学习算法 F 。如果普通模型回答“这张图是不是猫”，元学习关心的是“给我少量训练例子以后，怎样快速得到一个能回答问题的分类器”。

从工程角度看，元学习的动机至少有三类。

- **减少手工设计**：把网络结构、初始化、学习率、更新规则等原本靠人工或网格搜索决定的部分，改成可以从任务经验中学习。
- **提升少样本适应**：希望系统看到少量新任务样本后，就能快速得到可用模型。
- **复用跨任务经验**：如果过去见过许多任务，就不应该每遇到一个新任务都从零开始摸索学习方式。

¹ 视频画面时间区间：00:02:00–00:02:15。

不要把元学习等同于某一个具体算法

元学习不是 MAML、Matching Networks 或 few-shot classification 的同义词。它是一种问题设定：学习对象从“任务函数”上升为“学习算法”。不同论文可以学习初始化、学习率、优化器、距离函数、任务表示、网络结构，甚至学习整个更新规则。

1.1 本章小结

P55 的起点是调参困境：深度学习虽然自动学习权重，但大量“怎样学习”的选择仍然依赖人。元学习把这些选择放进可学习对象中，目标是让系统从一批任务中归纳出更好的学习算法。

2 从普通机器学习三步骤到元学习三步骤

课程用 Machine Learning 101 复习普通监督学习的三步骤：

1. 定义一个带未知参数的函数集合，例如神经网络 f_{θ} 。
2. 定义损失函数 $L(\theta)$ ，衡量某组参数在训练资料上的好坏。
3. 用优化方法找出让损失小的参数 θ^* 。

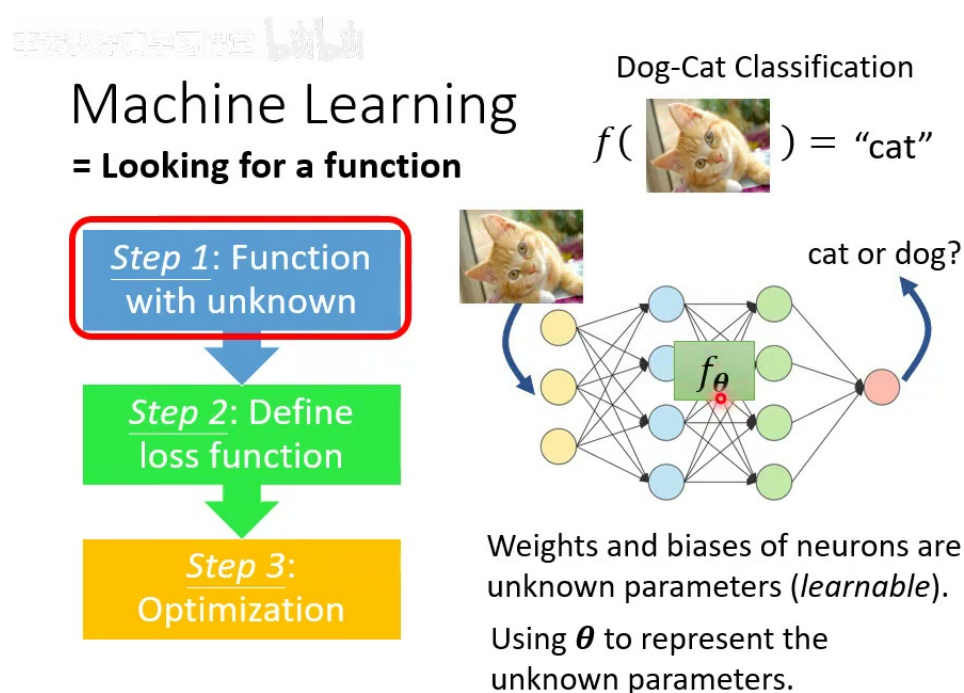


图 2: 普通机器学习三步骤：先定义带未知参数的函数集合，再定义 loss function，最后用优化方法找出最佳参数；图中以猫狗分类网络为例， θ 表示神经网络中的可学习权重和偏置。²

这套三步骤的关键是：未知对象是函数 f_{θ} 的参数 θ 。训练完成后得到的是一个具体函数 f_{θ^*} ，它可以对测试输入做预测。元学习把这个结构向上提升一层：未知对象不再只是分类器参数，而是学习算法本身。

²视频画面时间区间：00:05:45–00:06:00。

What is Meta Learning?

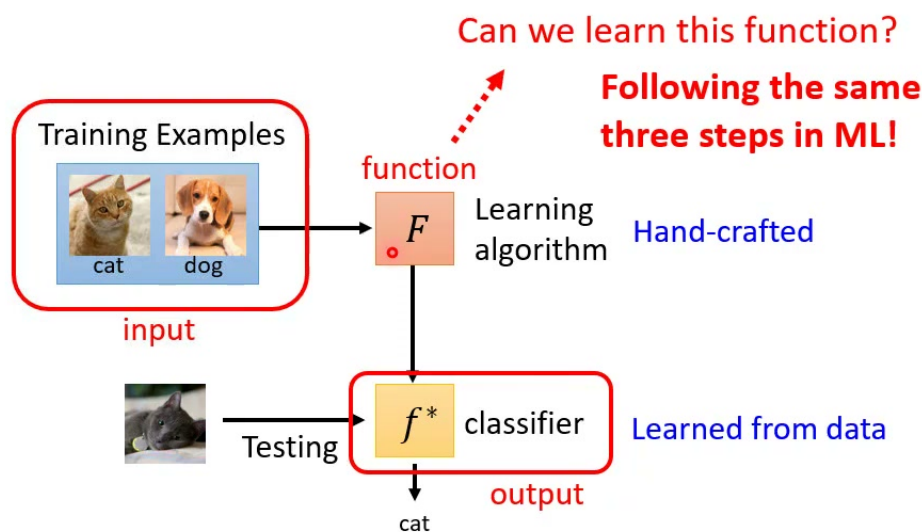


图 3: 元学习也沿用机器学习的三步骤，只是要找的函数变成学习算法 F ：它接收某个任务的训练样本，输出一个可用于该任务的分类器 f^* 。³

用符号写，可以把普通机器学习看成

$$\mathcal{D}^{\text{train}} \xrightarrow{\text{hand-crafted learning algorithm}} f_{\theta^*}.$$

其中学习算法通常由人设计，例如“用某个网络架构、交叉熵、Adam、固定学习率训练若干 epoch”。元学习则试图学习一个带参数的学习算法：

$$F_{\phi}(\mathcal{D}^{\text{train}}) = f^*.$$

符号说明如下。

$\mathcal{D}^{\text{train}}$ 某个任务提供的少量训练样本。

F_{ϕ} 带可学习组件 ϕ 的学习算法。

f^* 由学习算法针对当前任务产生的模型，例如分类器。

ϕ 学习算法内部待学习的部分，可以是初始化、度量、更新规则、网络结构等。

层级差异

普通机器学习学习的是模型参数 θ ；元学习学习的是学习算法参数 ϕ 。 θ 是某一个具体任务的模型参数， ϕ 则决定“看到任务训练样本以后如何产生 θ 或如何产生分类器”。这两个层级混在一起，后面的 support/query set、inner/outer loop 就会很容易看乱。

2.1 本章小结

普通机器学习和元学习共享同一个形式：定义候选空间、定义评价函数、做优化。区别在于候选对象不同。普通机器学习的候选对象是任务函数 f_{θ} ，元学习的候选对象是学习算法 F_{ϕ} 。

³视频画面时间区间：00:09:15–00:09:30。

3 Step 1: 学习算法里什么可以被学

元学习的第一步是定义“学习算法的集合”。课程用问题 What is learnable in a learning algorithm? 来展开：一个学习算法并不只是抽象动词“训练”，它包含许多组件。我们可以规定哪些组件由人工设定，哪些组件交给数据来学。



Meta Learning – Step 1

- What is **learnable** in a learning algorithm?

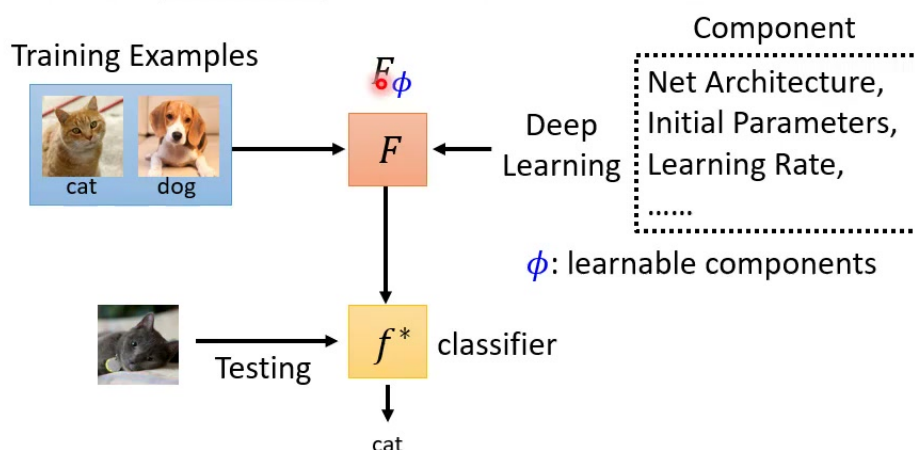


图 4: Step 1 关注学习算法中的可学习组件：网络架构、初始参数、学习率等都可以被视为 ϕ 的一部分；不同元学习方法的差异，常常来自“把哪些组件放进 ϕ ”。⁴

常见可学习组件包括：

- **初始参数**：学习一个好的起点，使新任务只需少量梯度更新即可适应。
- **学习率或更新步长**：不同层、不同参数甚至不同时间步可以有不同更新强度。
- **网络结构**：学习哪些模块、连接或结构适合一类任务。
- **距离或相似度度量**：在 few-shot classification 中，学习如何比较 support example 和 query example。
- **优化规则**：把“如何根据梯度更新参数”本身做成可学习程序。
- **任务表示**：学习如何把一个 task 的 support set 编码成可用于预测 query 的信息。

所以 F_ϕ 不是唯一形态。若 ϕ 是初始化参数，就得到 learning to initialize 的路线；若 ϕ 是度量函数，就接近 learning to compare；若 ϕ 是优化器，就接近 learning to optimize。课程此处先不进入具体算法，而是强调统一框架：这些方法都可以看成在定义一个带未知组件的学习算法。

⁴视频画面时间区间：00:11:45–00:12:00。

容易混淆的两种参数

ϕ 不是普通分类器的参数。分类器参数通常由某个任务内部的训练资料产生； ϕ 则来自许多 training tasks 的经验，用来决定以后遇到新任务时该怎样快速产生分类器。简单说， θ 属于 task 内部， ϕ 属于跨 task 的学习经验。

3.1 本章小结

元学习的 Step 1 是决定学习算法 F_ϕ 的候选空间。它可以学习初始化、学习率、结构、度量、优化规则或任务表示。不同元学习分支的本质差异，往往就是 ϕ 代表的东西不同。

4 Step 2: 如何评价一个学习算法

定义了 F_ϕ 之后，第二步要定义损失函数 $L(\phi)$ 。在普通机器学习里，loss 衡量某个模型在训练样本上的预测错误；在元学习里，loss 衡量某个学习算法在一批任务上的表现。也就是说，评价对象从“一个模型”变成“一个会产生模型的算法”。

设有 N 个 training tasks:

$$\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_N.$$

每个任务都有自己的训练部分和测试部分:

$$\mathcal{T}_n = (\mathcal{D}_n^{\text{train}}, \mathcal{D}_n^{\text{test}}).$$

对第 n 个任务，学习算法先看该任务的训练资料:

$$f_n = F_\phi(\mathcal{D}_n^{\text{train}}).$$

然后用该任务的测试资料评价 f_n :

$$\ell^n(\phi) = \sum_{(x,y) \in \mathcal{D}_n^{\text{test}}} \text{CE}(f_n(x), y).$$

最后把所有 training tasks 的评价加总:

$$L(\phi) = \sum_{n=1}^N \ell^n(\phi).$$

符号说明如下。

$\ell^n(\phi)$ 第 n 个 task 上，学习算法 F_ϕ 产出的模型在该 task 测试部分上的错误。

$L(\phi)$ 跨所有 training tasks 的总损失，用来评价学习算法整体好坏。

N 用来训练学习算法的任务数量，而不是单一任务里的样本数量。

Meta Learning – Step 2

Task 1

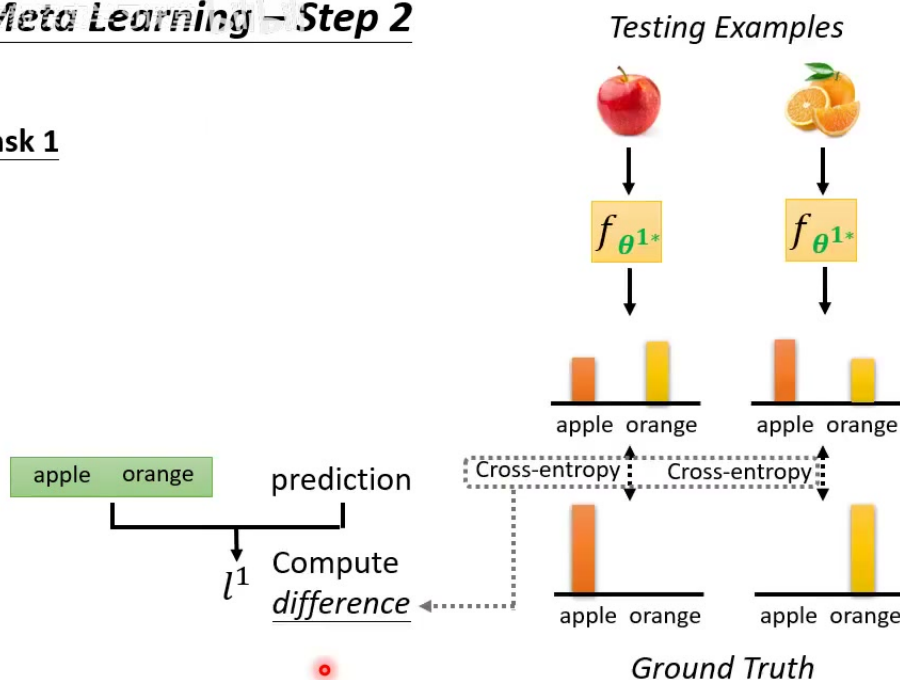


图 5: Step 2 的核心: 同一个学习算法 F_ϕ 会在多个 training tasks 上分别产出分类器, 再用各 task 的测试部分计算 loss; 所有 task loss 加总后才是 $L(\phi)$ 。⁵

这一步很反直觉: 在每个 training task 内部, 计算 meta-loss 时确实会用到这个 task 的 testing examples。原因是我们不是在训练该 task 的最终分类器, 而是在训练学习算法; training task 的 testing examples 正好提供“这个学习算法是否真的会学”的评价信号。

为什么 training task 可以有 testing examples

对元学习而言, training task 整体都是训练资料。它内部再切成 train/test, 是为了模拟未来新任务的使用方式: 学习算法只能先看少量 train examples, 然后必须在未见过的 examples 上表现好。因此, training task 的 test split 用来训练 F_ϕ ; 真正不能碰的是最终 testing task 的资料。

不要把 task 的 test split 和最终测试集混为一谈

“Task 内部的 testing examples”与“整个 meta-learning 实验的 testing tasks”是两层概念。前者在 meta-training 中可以用于计算 $L(\phi)$; 后者用于评估最终学到的学习算法, 训练时不能使用。

4.1 本章小结

元学习的 Step 2 是定义学习算法的 loss。它不是看某个固定分类器在一个数据集上错多少, 而是看同一个学习算法在许多 training tasks 中, 能否用少量训练样本产生能泛化到该 task 测试部分的模型。

⁵视频画面时间区间: 00:19:45–00:20:00。

5 Step 3: 优化学习算法本身

第三步是找出让 $L(\phi)$ 最小的学习算法参数：

$$\phi^* = \arg \min_{\phi} L(\phi).$$

如果 $L(\phi)$ 对 ϕ 可微，就可以使用熟悉的梯度下降：

$$\phi \leftarrow \phi - \eta \nabla_{\phi} L(\phi).$$

若 $L(\phi)$ 不可微，仍然可以把它当作黑箱优化问题，使用强化学习、演化算法或其他 derivative-free optimization 方法。



Meta Learning – Step 3

- Loss function for learning algorithm $L(\phi) = \sum_{n=1}^N l^n$
- Find ϕ that can minimize $L(\phi)$ $\phi^* = \arg \min_{\phi} L(\phi)$
- Using the optimization approach you know
 - If you know how to compute $\partial L(\phi)/\partial \phi$
Gradient descent is your friend.
 - What if $L(\phi)$ is not differentiable?
Reinforcement Learning / Evolutionary Algorithm

图 6: Step 3 把学习算法参数 ϕ 当作优化对象：若能计算 $\partial L(\phi)/\partial \phi$ ，就可用梯度下降；若不可微，也可考虑强化学习或演化算法等黑箱优化方法。⁶

优化完成后得到 F_{ϕ^*} ，也就是课程中的 learned learning algorithm。未来遇到一个新任务时，不再从手工学习算法开始，而是使用 F_{ϕ^*} ：

$$f_{\text{new}} = F_{\phi^*}(\mathcal{D}_{\text{new}}^{\text{train}}).$$

如果元学习成功， F_{ϕ^*} 会把过去 training tasks 中积累的经验压缩进 ϕ^* ，使新任务适应更快、所需标注更少，或超参数搜索更少。

元学习训练本身可能很贵

元学习常常把“许多 task 内部的训练过程”嵌进一个更外层的优化中。即使每个 task 都很小，总成本也可能很高，因为每次更新 ϕ 都要模拟若干任务上的学习与评估。这也是为什么元学习论文常特别关注效率、近似梯度、任务采样和 inner loop 长度。

⁶ 视频画面时间区间：00:23:30–00:23:45。

5.1 本章小结

元学习的 Step 3 是在 task 层级上优化 F_ϕ 。普通机器学习优化 θ ，得到一个任务模型；元学习优化 ϕ ，得到一个可复用于新任务的学习算法。只要能定义 $L(\phi)$ ，就可以用梯度法或黑箱优化法寻找 ϕ^* 。

6 Few-shot 框架中的任务、资料与循环

课程接着把抽象框架落到 few-shot learning 语境。Few-shot learning 是目标：希望模型只看少量标注样本就能解决新任务。Meta learning 是常见手段：通过许多 training tasks 训练一个学习算法，使它在未来 testing task 上只需少量样本就能适应。

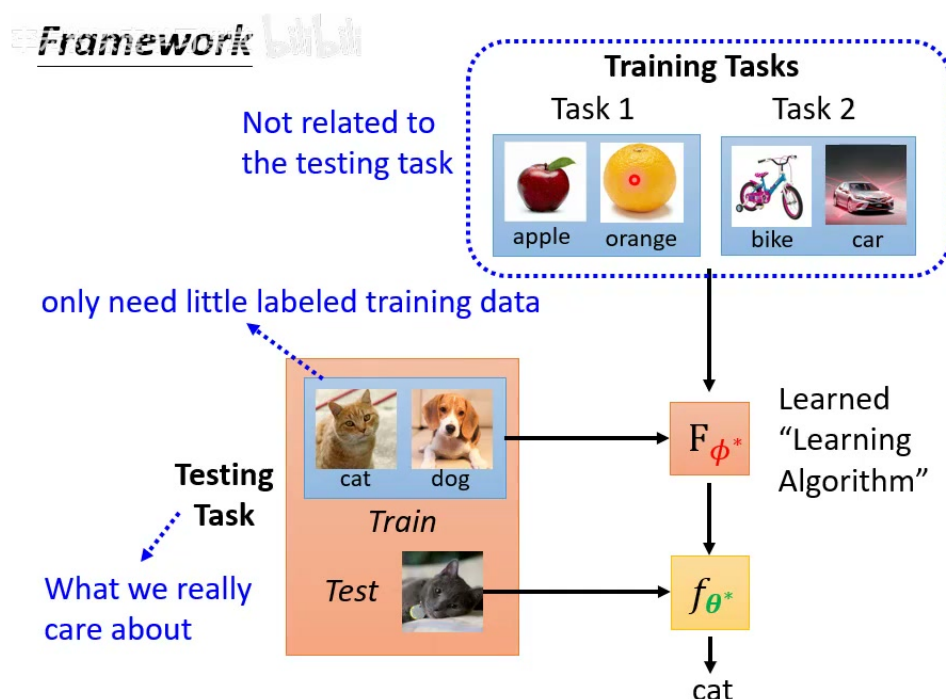


图 7: 元学习框架中的 training tasks 与 testing task: 上方 training tasks 用来学习 F_{ϕ^*} ; 真正关心的是左下方新的 testing task, 它只提供少量 labeled training data, 并要求模型在该 task 的 test examples 上表现好。⁷

这个图中最重要的是边界: training tasks 与 testing task 可以来自同一类任务分布, 但 testing task 本身不能参与训练 F_ϕ 。如果把 testing task 的 train split 也拿来训练 ϕ , 就会破坏 few-shot/meta-learning 的评估设定。

6.1 Support set 与 query set

为了避免“训练资料”和“测试资料”这两个词在不同层级上混乱, meta-learning 文献常把 task 内部的训练部分叫 support set, 把 task 内部的测试部分叫 query set。

⁷ 视频画面时间区间: 00:27:45–00:28:00。

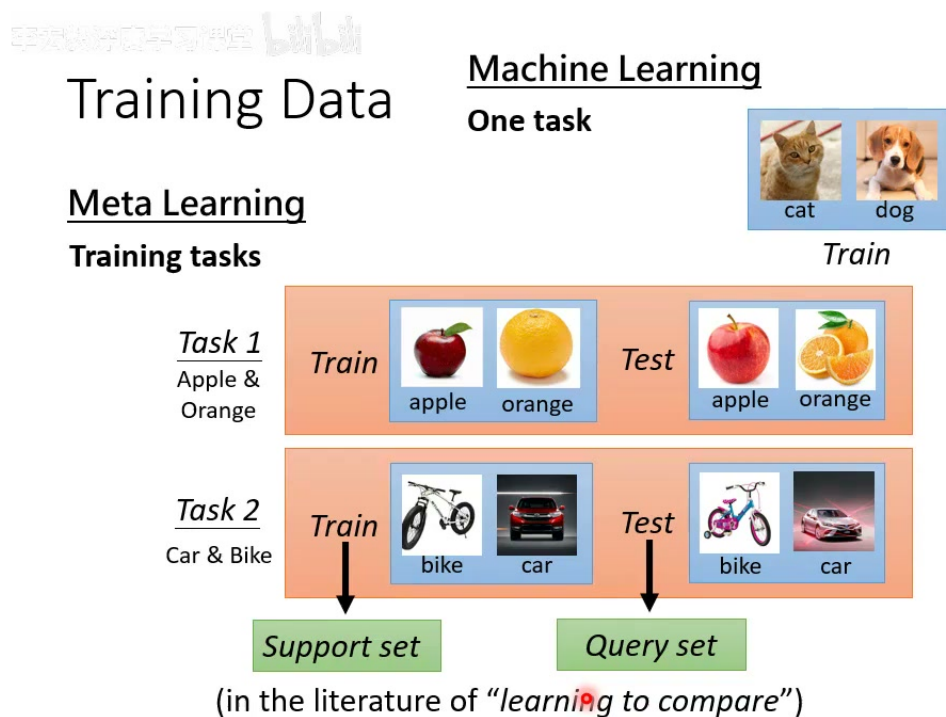


图 8: 在元学习文献中，一个 task 内部常被切成 support set 与 query set: support set 用来让学习算法产生该 task 的模型，query set 用来计算这个 task 对学习算法的评价。⁸

这个命名非常有用：

- **Support set:** 给学习算法看的少量标注样本，负责“支持”当前 task 的适应。
- **Query set:** 当前 task 中用于评价的样本，负责检验由 support set 产生的模型是否泛化。
- **Training tasks:** 许多带 support/query 切分的任务，用来训练 F_ϕ 。
- **Testing task:** 最终评估用的新任务，只能用它 support set 做适应，再在 query/test 上评估。

命名不统一会导致的误解

如果说“用 testing examples 训练”，听起来像数据泄漏；但如果说“用 training task 的 query set 计算 meta-loss”，含义就清楚得多。元学习中最危险的混乱，往往不是公式难，而是层级命名没有区分。

6.2 Within-task 与 across-task

课程进一步区分两种 training：

- **Within-task training:** 在单个 task 内，根据 support set 得到该 task 的模型。
- **Across-task training:** 跨许多 training tasks，学习出一个更好的学习算法 F_ϕ 。

⁸视频画面时间区间：00:30:45–00:31:00。

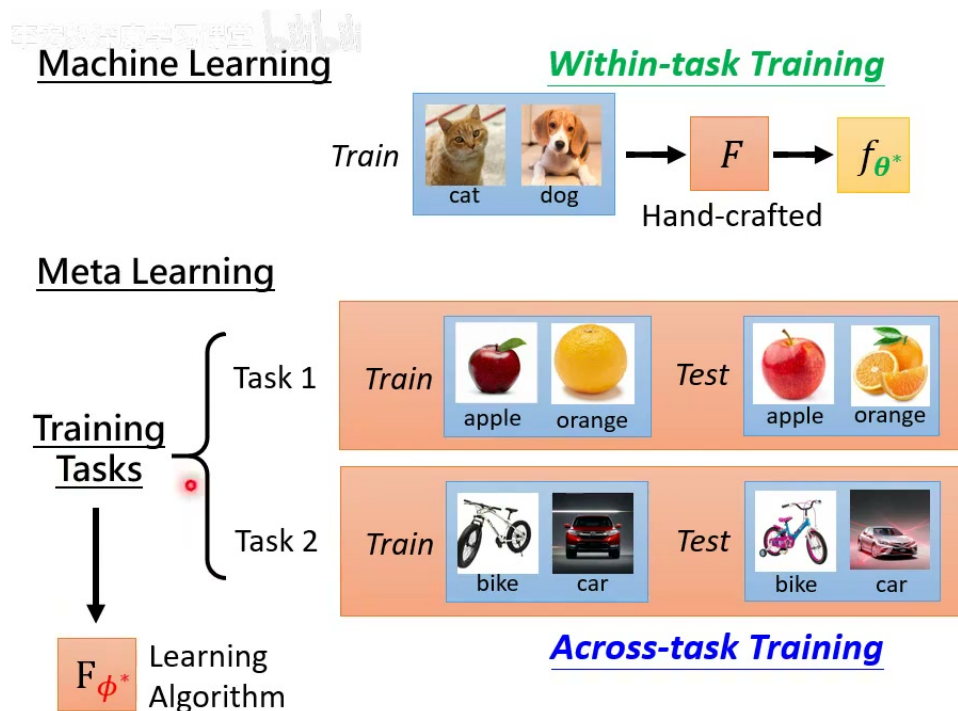


图 9: 普通机器学习只包含一个 task 内部的训练; 元学习同时包含 across-task training 和 within-task training: 先跨任务学习学习算法, 再在每个新任务内部用少量 support examples 产生分类器。⁹

这一区分解释了元学习为什么经常看起来像“双层训练”。内层是 task 内部的模型适应, 外层是学习算法本身的更新。对于 learning to initialize 类方法, 内层可能是几步梯度下降, 外层则根据 query loss 更新初始化参数。

6.3 Loss 层级: 样本求和与任务求和

普通机器学习的 loss 往往是在一个任务的训练样本上求和:

$$L(\theta) = \sum_{k=1}^K e_k.$$

元学习则多一层任务求和:

$$L(\phi) = \sum_{n=1}^N \ell^n, \quad \ell^n = \sum_{m=1}^{M_n} e_m^n.$$

其中 e_m^n 是第 n 个 task 的第 m 个 query example 的错误, ℓ^n 是一个 task 的 query loss, $L(\phi)$ 是跨 training tasks 的总损失。

⁹视频画面时间区间: 00:32:00–00:32:15。

Loss

Machine Learning

$$L(\theta) = \sum_{k=1}^K e_k \rightarrow \text{Sum over training examples in one task}$$

Meta Learning

$$L(\phi) = \sum_{n=1}^N \ell^n \rightarrow \text{Sum over testing examples in one task}$$

图 10: 普通机器学习的 loss 对单一任务的训练样本求和; 元学习的 loss 先在每个 task 的 query examples 上求和得到 ℓ^n , 再跨 training tasks 求和得到 $L(\phi)$ 。¹⁰

这个层级也说明为什么元学习需要 task distribution。若只有一个任务, 就很难判断一个学习算法是否真的“会学习”; 它可能只是记住了一个任务的特殊性。多个 tasks 提供了泛化压力, 让 F_ϕ 学到跨任务可复用的规律。

6.4 Episode 与 inner/outer loop

在 few-shot 元学习中, 一次“从 support set 适应, 再在 query set 上评价”的流程常被称为一个 episode。Across-task training 会采样许多 episodes, 用它们来更新 F_ϕ 。

¹⁰视频画面时间区间: 00:34:45–00:35:00。

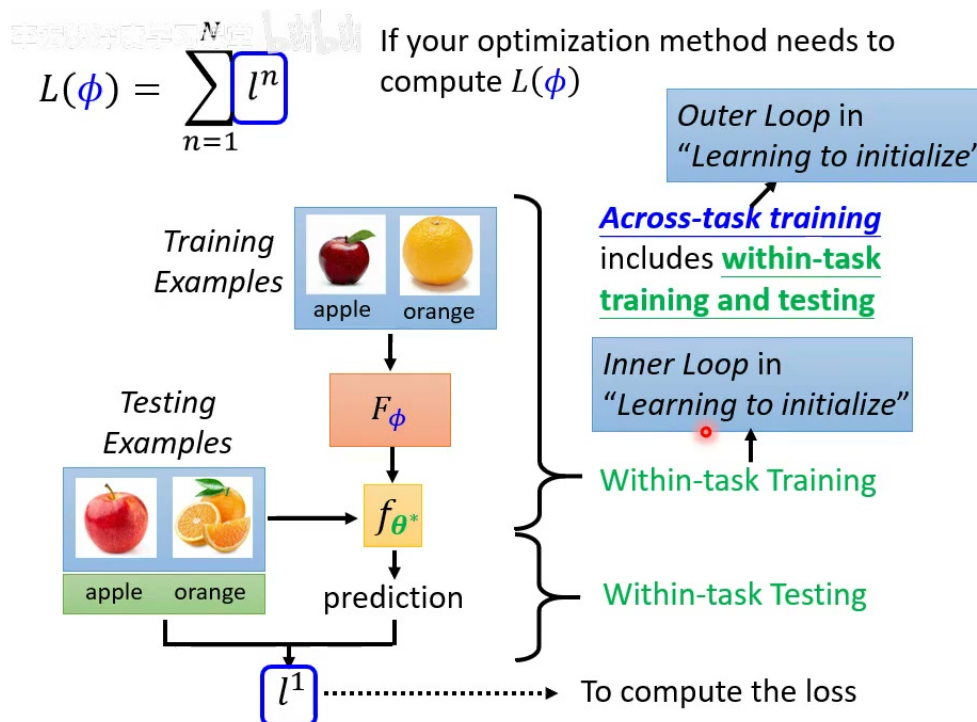


图 11: 在需要显式模拟任务内训练的元学习方法中, 常把 across-task training 称为 outer loop, 把 within-task training 称为 inner loop; 但这个叫法最适合 learning to initialize, 不一定适用于所有元学习方法。¹¹

课程特别提醒: outer loop 与 inner loop 的说法并非所有元学习方法都必须采用。它最自然地出现在 learning to initialize 一类方法中, 因为这些方法确实有一个反复执行的任务内优化过程。但如果学习算法不是迭代式的, 例如一次前向传播就输出分类器或度量结果, 那么 inner loop 的概念就不一定恰当。

把 few-shot episode 看成一次模拟考试

Support set 相当于给学生看的复习资料, query set 相当于同一个 task 的考试题。Meta-training 不断采样许多这样的模拟考试, 让学习算法学会“怎样利用少量资料准备考试”。最终 testing task 则是全新的考试, 训练时不能提前看到。

6.5 本章小结

Few-shot 语境下, 元学习最容易混乱的是层级。Task 内部有 support/query; task 之间有 training/testing; 训练过程又有 within-task 与 across-task。理解这些层级后, $L(\phi)$ 的结构就很自然: 它衡量学习算法在许多任务中“看 support、答 query”的整体能力。

7 总结与延伸

课程最后强调: 你在普通机器学习中熟悉的许多概念, 通常也能迁移到元学习, 只是层级要上移。普通机器学习担心 overfitting on training examples; 元学习担心 overfitting on training tasks。普通机器学习可以增加训练样本; 元学习可以增加 training tasks。普通机器学习有 data augmentation; 元学

¹¹ 视频画面时间区间: 00:37:15–00:37:30。

习也可以做 task augmentation。普通机器学习需要开发集调超参数；元学习也可以设置 development tasks 来选择算法与超参数。



Meta Learning v.s ML

- What you know about ML can usually apply to meta learning
 - Overfitting on training tasks
 - Get more training tasks to improve performance
 - Task augmentation
 - There are also hyperparameters when learning a learning algorithm
 - Development task ☺

图 12: Meta Learning 与普通 ML 的类比：普通 ML 中关于过拟合、增加训练数据、数据增强、超参数与开发集的经验，通常可以上移到 task 层级，在 meta-learning 中变成 training tasks、task augmentation 与 development tasks。¹²

把 P55 压缩成一句话：元学习不是要推翻机器学习，而是把机器学习的三步骤递归地用在“学习算法”上。普通机器学习的问题是：

如何从资料中找出好函数？

元学习的问题是：

如何从许多任务中找出一个好学习算法？

因此，本讲的核心对应关系可以整理如下。

概念	普通机器学习	元学习
待学习对象	任务函数 f_θ	学习算法 F_ϕ
输入资料	一个任务的训练样本	多个 training tasks
输出结果	一个具体模型 f_{θ^*}	一个 learned learning algorithm F_{ϕ^*}
评价方式	模型在样本上的 loss	学习算法在多个 task query sets 上的 loss
泛化目标	泛化到同一任务的新样本	泛化到新 task

¹²视频画面时间区间：00:38:30-00:38:45。

P55 最重要的概念压缩

元学习的“资料”不是单个样本，而是一批任务；元学习的“模型”不是单个分类器，而是学习算法；元学习的“泛化”不是只到新样本，而是到新任务。把这三点抓住，后续看 MAML、metric-based meta-learning、learning to optimize 等方法时，就能知道它们究竟在学习哪一层。

从后续学习角度看，本讲提供的是统一语法。之后遇到具体方法时，可以逐一追问四个问题：

1. 它把学习算法 F_ϕ 里的哪一部分设为可学习？
2. 它如何构造 task、support set 与 query set？
3. 它的 meta-loss $L(\phi)$ 怎样计算，是否需要反传穿过 inner loop？
4. 它最终希望在什么类型的新 task 上快速适应？

这些问题比直接背方法名更有用。因为元学习方法很多，但它们大多都能放回 P55 的三步骤：定义 F_ϕ ，定义 $L(\phi)$ ，优化 ϕ 。

7.1 本章小结

P55 建立了元学习的基本框架：普通机器学习找函数，元学习找学习算法；普通 loss 在样本层级求和，meta-loss 在 task 层级求和；普通训练关注一个任务内的模型参数，meta-training 关注跨任务可复用的学习经验。后续所有具体元学习算法，都可以先放进这个框架中定位。